CS310 Natural Language Processing
自然语言处理
Lecture 06: Pretraining Large Language Models

Instructor: Yang Xu

主讲人：徐炀

xuyang@sustech.edu.cn

Overview

- Data
- Model Architectures
- Pretraining Process

Overview

- **Data**
  - **Pretraining Tasks**
  - **Data**
  - **Tokenization**
- Model Architectures
- Pretraining Process

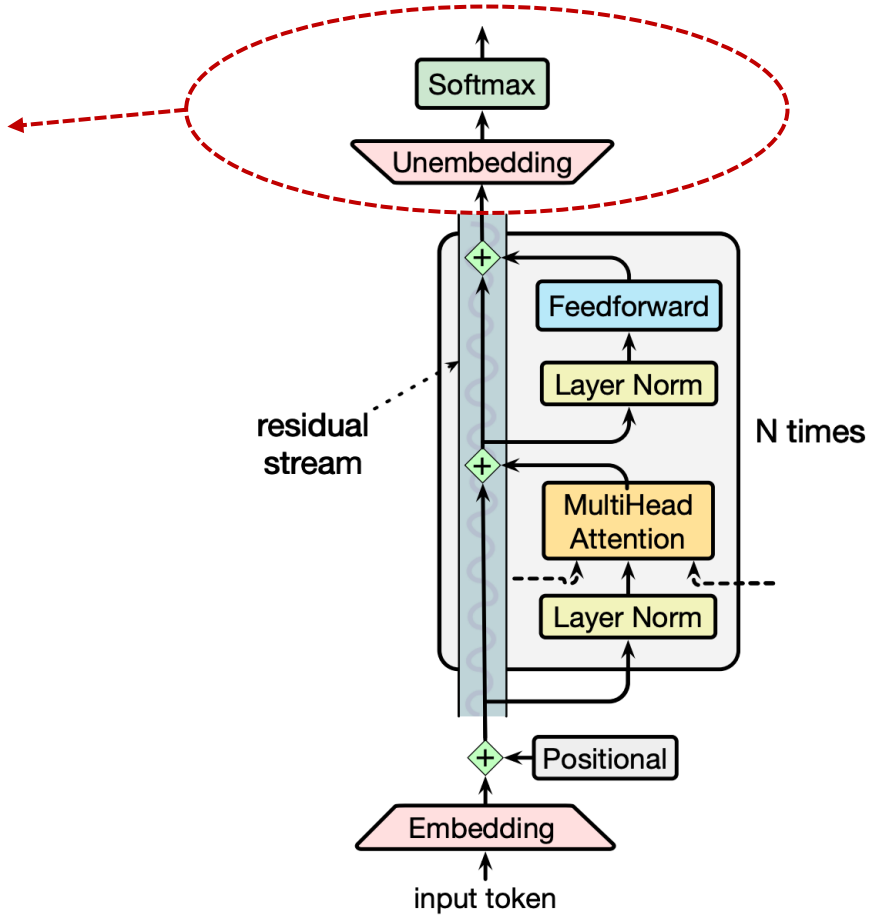
What is pretraining?

- "Pretraining" is a term used in machine learning
- Prefix "pre-" means "*happening before*…"
- The initial, resource-intensive phase in ML where a model is trained on ***massive***, ***unlabeled*** data to learn broad patterns …
  - Applied not just to LLMs, but also vision models and other …

- Outcome of pretraining: A model with *general knowledge* that requires less, task-specific data to become highly proficient through fine-tuning.

Learning tasks for pretraining LLMs

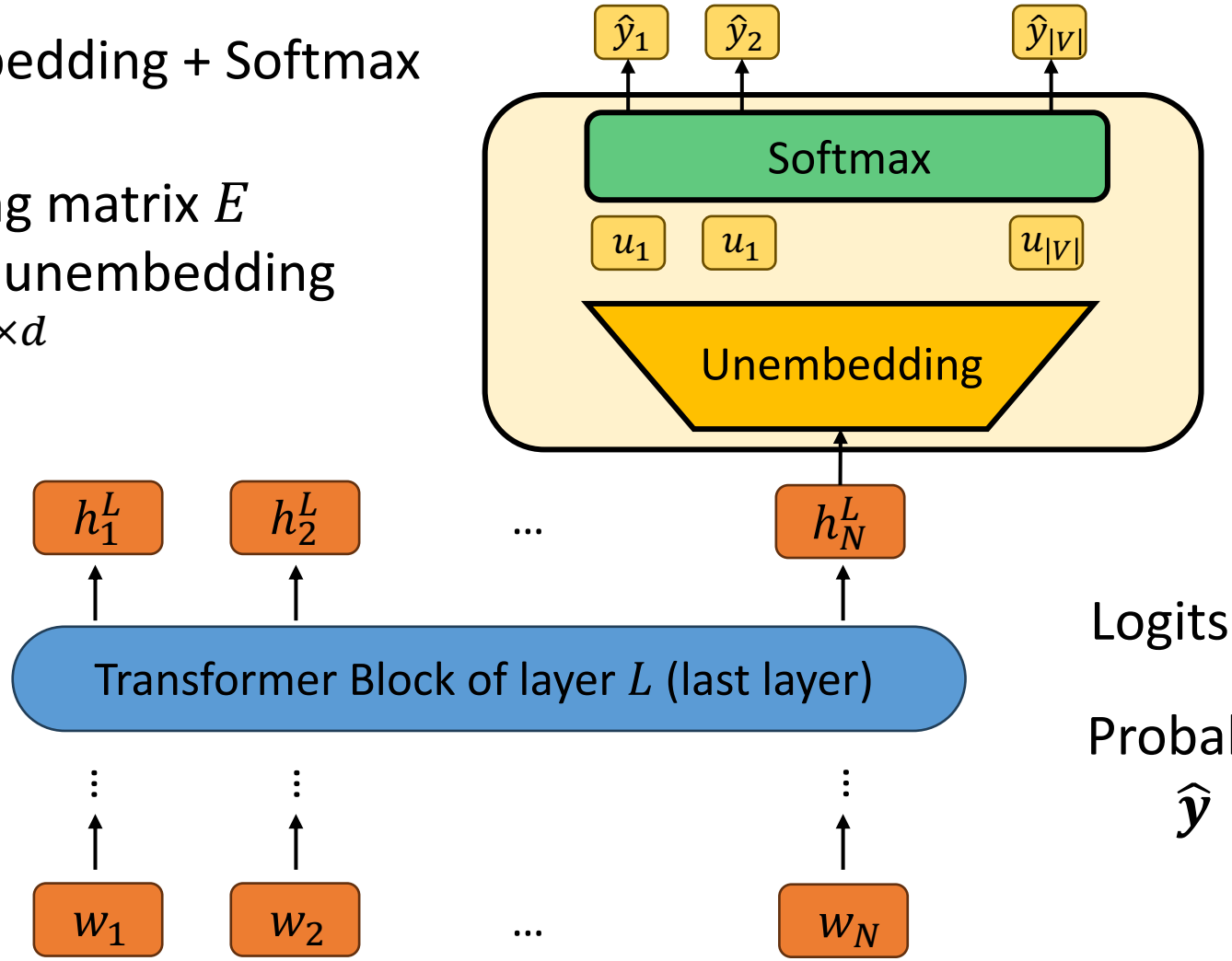
- Task: Predicting the next token.
- Using the language modeling head (**LMHead**)
- LMHead = Unembedding + Softmax



LM Head

- **LMHead** = Unembedding + Softmax

Use the embedding matrix E transposed as the unembedding matrix: $\boldsymbol{E}^\top \in \mathbb{R}^{|V| \times d}$



Logits: $\boldsymbol{u} = \boldsymbol{h}_N^L \cdot \boldsymbol{E}^\top$

Probability distribution: $\hat{\boldsymbol{y}} = \text{Softmax}(\boldsymbol{u})$

Use LM Head's output to predict

- At each position t, take as input the word sequence $w_{1:t}$, compute a probability distribution $\hat{\boldsymbol{y}}_t$, and then compute the **cross-entropy** loss between this distribution and the actual next word $\boldsymbol{y}_t[w_{t+1}]$

$\boldsymbol{y}_t$ is a one-hot vector of length $|V|$:

- $\boldsymbol{y}_t[w = w_{t+1}] = 1$
- $\boldsymbol{y}_t[w \neq w_{t+1}] = 0$

Cross-entropy loss:

- $L_{CE} = -\sum_{w \in V} \boldsymbol{y}_t[w] \log \hat{\boldsymbol{y}}_t = -\log \hat{\boldsymbol{y}}_t[w_{t+1}]$
- Then average over entire sequence: $\frac{1}{T}\sum_{t=1}^{T} L_{CE}$

Sequence length T is large:

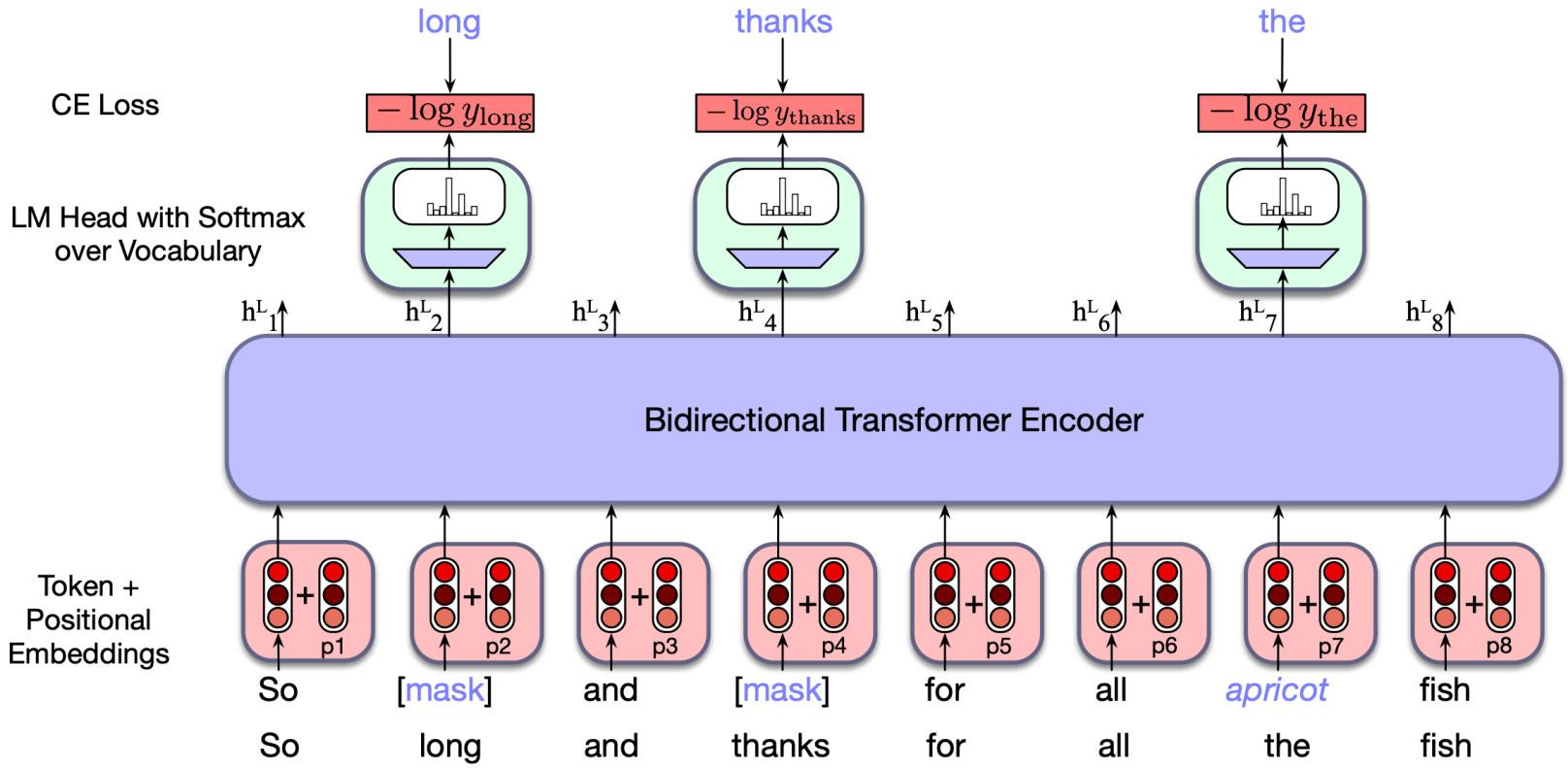
T = 2048 or 4096 for GPT3 or GPT3.5

GPT4: 8192; GPT4 Turbo: 128000

Source: https://www.scriptbyai.com/token-limit-openai-chatgpt

Other pretraining tasks

- **Masked Language Modeling (MLM)**: Predict the masked tokens in a sequence, with bidirectional attentions. (used in BERT and its variants)



$$\mathcal{L}_{\text{MLM}} = -\frac{1}{|M|}\sum_{i\in M}\log P(x_i|\boldsymbol{z}_i)$$

$$= -\frac{1}{3}[\log P(long|\boldsymbol{z}_2) + \log P(thanks|\boldsymbol{z}_4) + \log P(the|\boldsymbol{z}_7)]$$

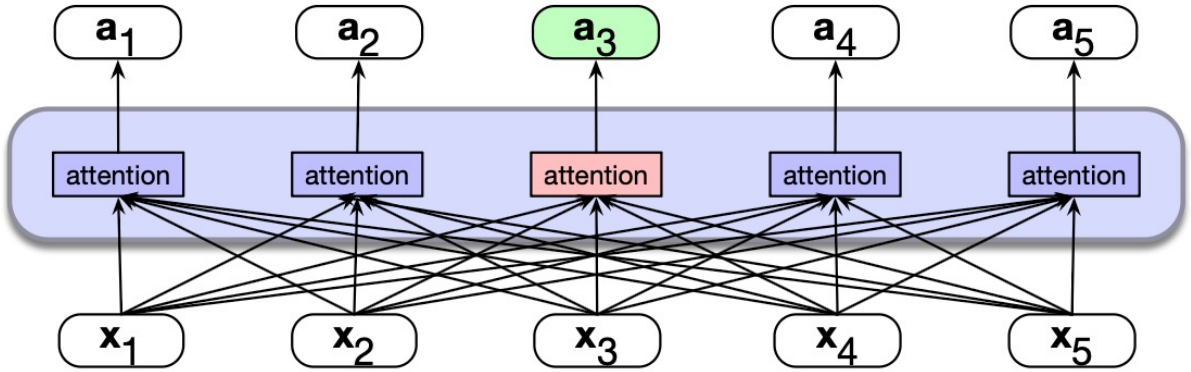
The masked tokens $M = \{long, thanks, the\}$ have been sampled:

long ⇒ [MASK]

thanks ⇒ [MASK]

the ⇒ *apricot*

MLM and bidirectional self-attention



$$\boldsymbol{QK}^\top$$

N	q1·k1	q1·k2	q1·k3	q1·k4
	q2·k1	q2·k2	q2·k3	q2·k4
	q3·k1	q3·k2	q3·k3	q3·k4
	q4·k1	q4·k2	q4·k3	q4·k4

N

Different from causal self-attention, bidirectional self-attention allows the attention mechanism to range over the entire input, i.e., can look at future tokens.

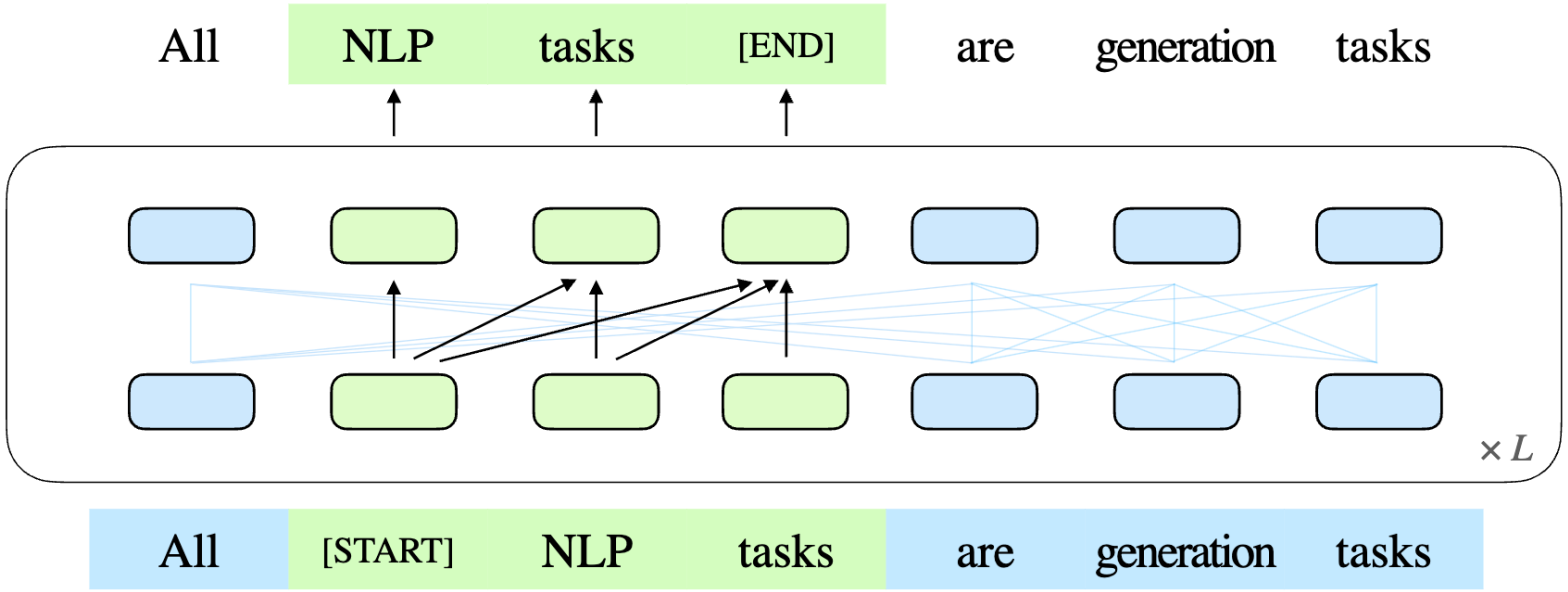
$$\boldsymbol{A} = \text{softmax}\left(\cancel{\text{mask}}\left(\frac{\boldsymbol{QK}^\top}{\sqrt{d_k}}\right)\right)\boldsymbol{V}$$

$$\boldsymbol{A} = \text{softmax}\left(\frac{\boldsymbol{QK}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}$$

Other pretraining tasks

Du et al., 2022, https://aclanthology.org/2022.acl-long.26.pdf

- **General Language Modeling (GLM)**: Randomly blank out spans of tokens and train the model to sequentially reconstruct the spans.



Data for pretraining

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

- Sources of data: **General** text + **Specialized** text
- General text:
  - *Webpages*: CommonCrawl, Wikipedia, …
  - *Conversation text*: PushShift.io Reddit corpus, …
  - *Books*: The Pile (containing Books3, Bookcorpus2), Gutenburg, …

- *Webpages* ⇒ diverse knowledge and enhance generalization capabilities
- *Conversation* ⇒ enhance conversational competence and QA performances
- *Books* ⇒ provide formal long text, beneficial for learning linguistic knowledge and long-term dependencies

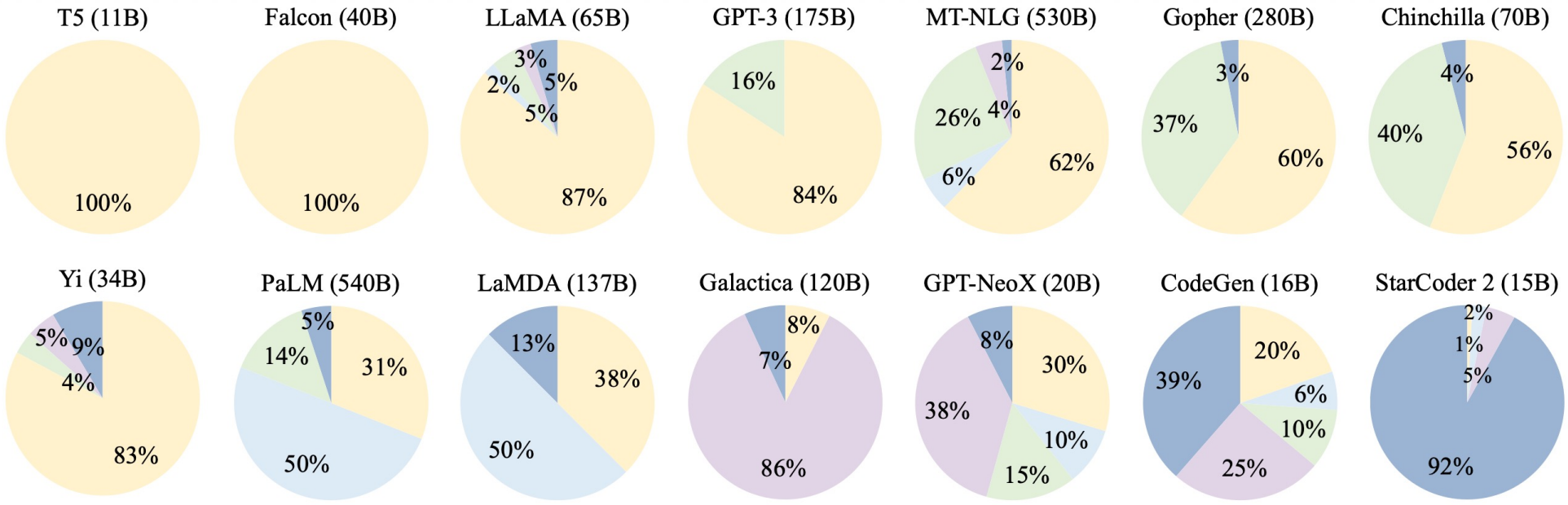
Data for pretraining

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

- Specialized text:
  - *Multilingual text*:
    To enhance multilingual abilities in understanding and generation.
    BLOOM and PALM (46 and 122 languages)
    FLM (Chinese and English in equal proportions)
  - Scientific text:
    To enhance the understanding of scientific knowledge for LLMs.
    E.g., arXiv papers, science textbooks, math webpages
  - *Code*:
    To enable LLMs to generate high-quality and accurate programs.
    E.g., public software repositories such as Github

Data recipes examples

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

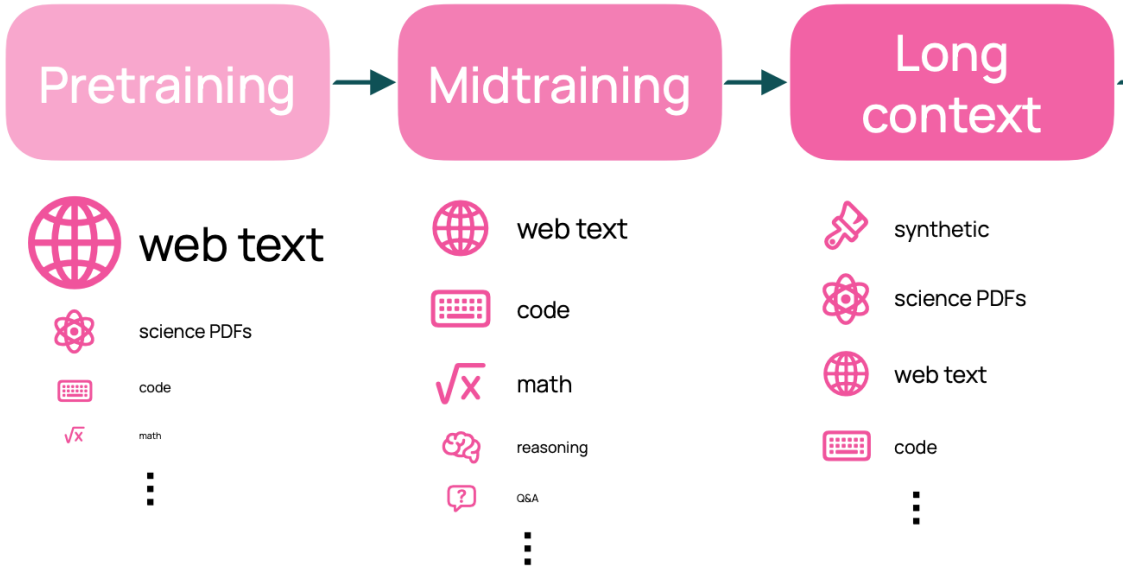


Webpages	C4 (800G, 2019), OpenWebText (38G, 2023), Wikipedia (21G, 2023)
Conversation Data	the Pile - StackExchange (41G, 2020)
Books & News	BookCorpus (5G, 2015), Gutenberg (-, 2021), CC-Stories-R (31G, 2019), CC-NEWES (78G, 2019), REALNEWs (120G, 2019)
Scientific Data	the Pile - ArXiv (72G, 2020), the Pile - PubMed Abstracts (25G, 2020)
Code	BigQuery (-, 2023), the Pile - GitHub (61G, 2020)

Data recipes examples

Reference: Olmo3 Technical Report, https://arxiv.org/pdf/2512.13961

- Olmo 3 recipes



Base Data: `Pretrain: Dolma 3 Mix` `Midtrain: Dolma 3 Dolmino Mix` `Long-ctx: Dolma 3 Longmino Mix`

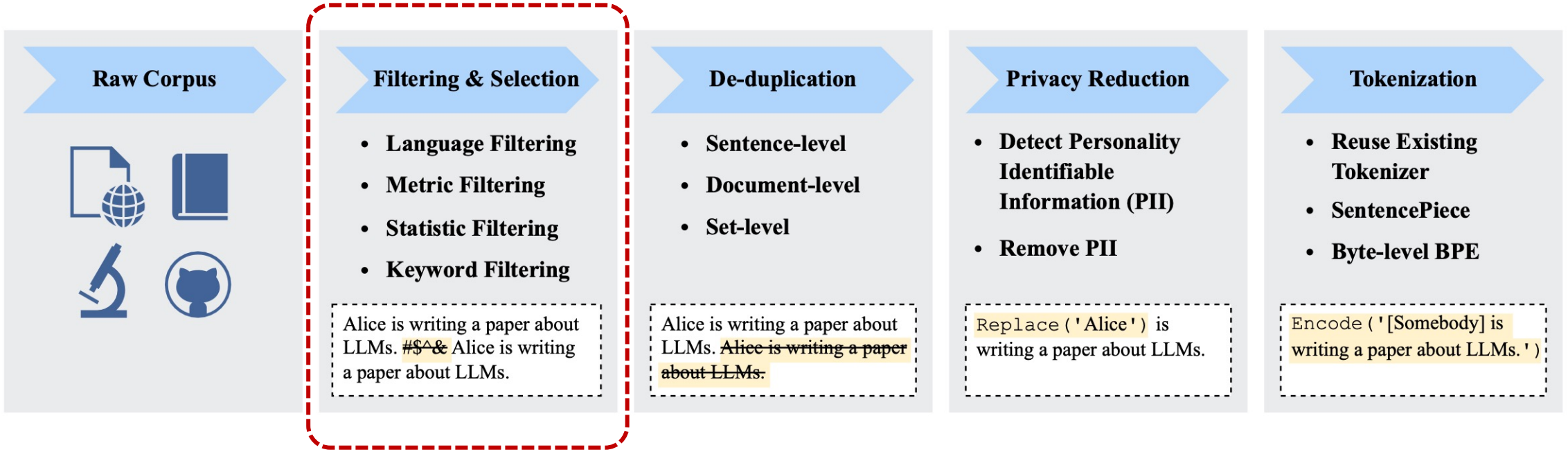
Examples of datasets

- **E.g., Common Crawl**: A series of snapshots of the entire web with billions of webpages
- E.g., Colossal Clean Crawled Corpus (C4; Raffel et al., 2020): 156 billion tokens
  - Mostly patent text documents, Wikipedia, and news sites (according to Dodge et al., 2021)
- E.g., MNBVC (Massive Never-ending BT Vast Chinese corpus)
  超大规模中文语料集

- Needs a LOT of data preprocessing work!

Data preprocessing

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

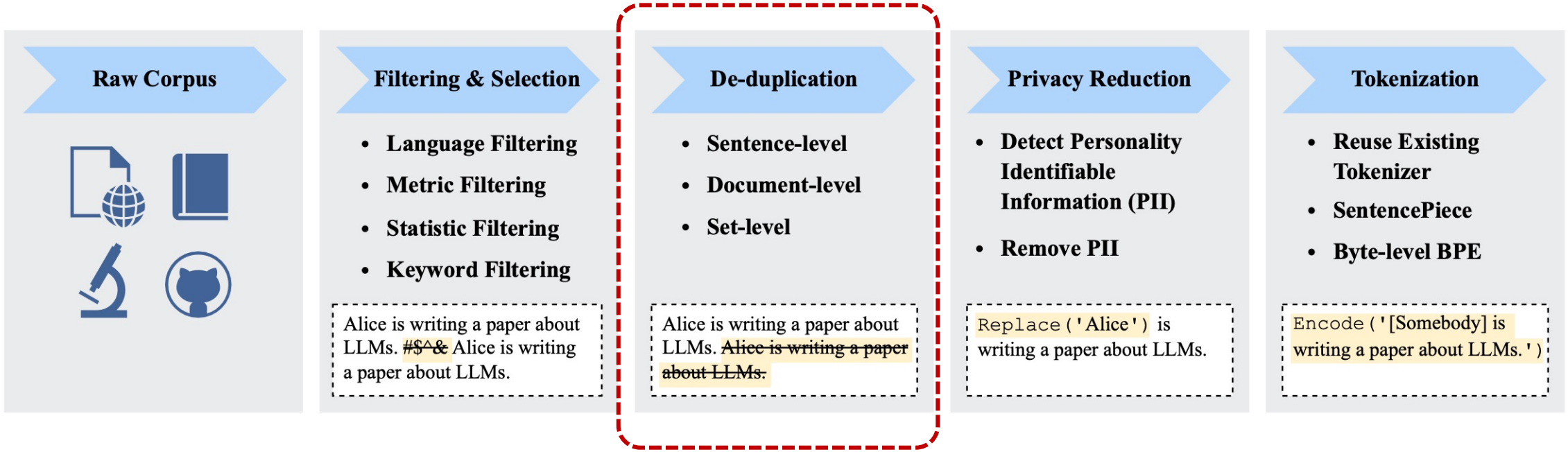


Filtering and Selection: Remove low-quality data

- Language-based: e.g., EN-CN
- Metric-based: e.g., perplexity
- Statistic-based: e.g., sentence length

Data preprocessing

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

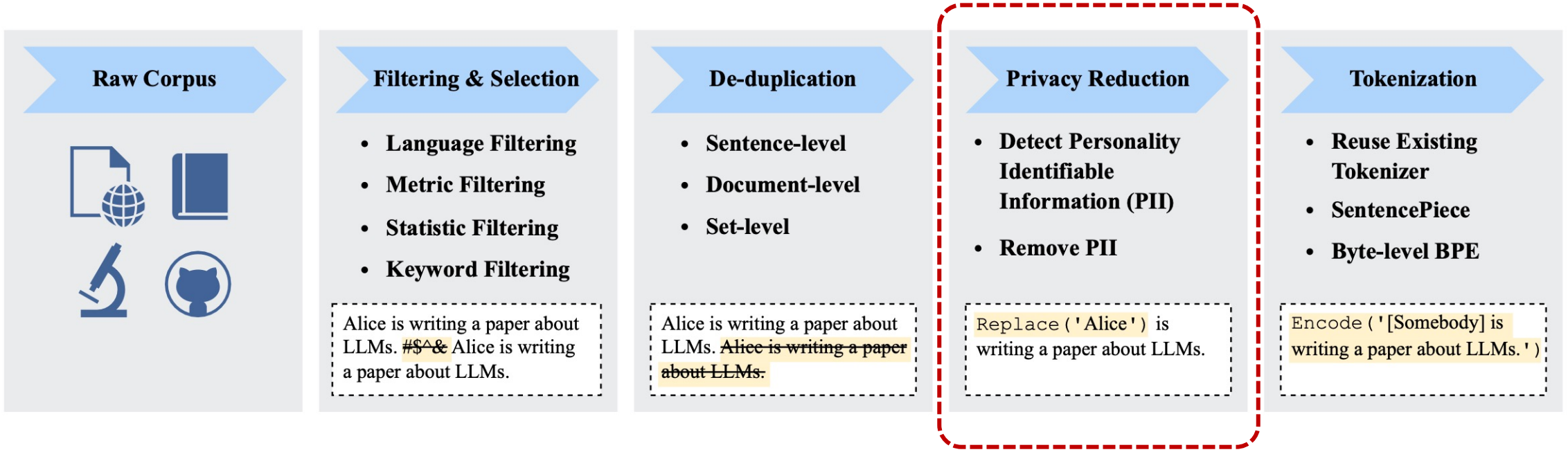


Duplicate data in corpus may cause the training process to become unstable, and reduce the diversity of language models.

- Sentence level, document-level, dataset-level.

Data preprocessing

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223

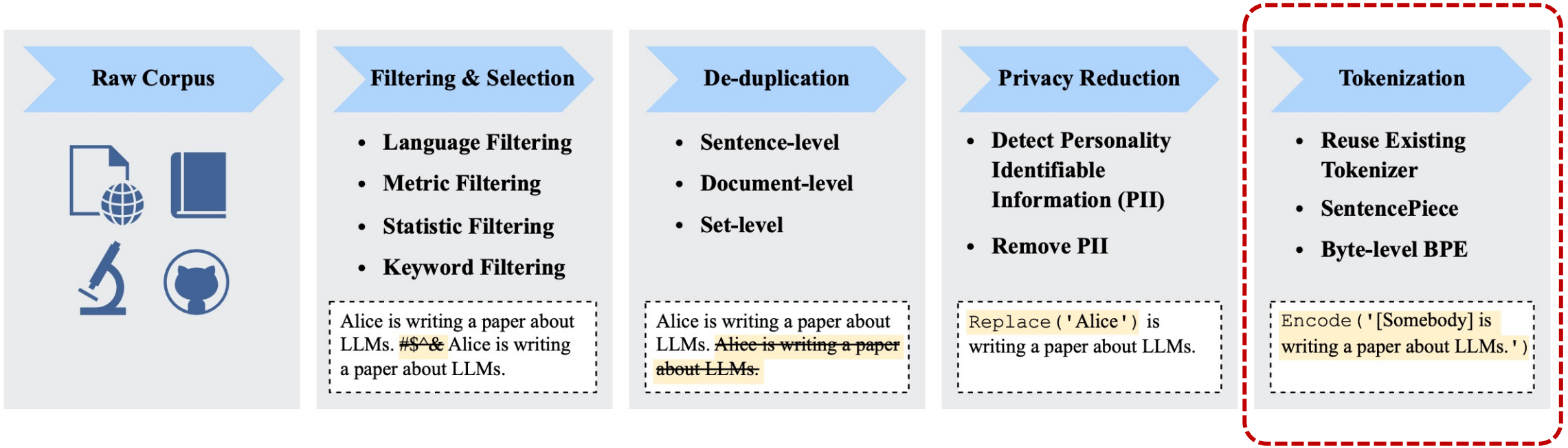


Remove personally identifiable information (PII)

- Key word spotting: names, addresses, phone numbers …

Data preprocessing

Reference: A Survey of Large Language Models, https://arxiv.org/pdf/2303.18223



Segment raw text into sequences of individual tokens, which will be subsequently used as the inputs of LLMs.

Tokenization 词元化

- Tokenization: raw string ⇒ sequence of tokens
- Token (词元): the basic information processing unit of LLMs

- Traditional NLP uses vocabulary-based tokenization
  - Problems: Too many low-frequency tokens, out-of-vocabulary (OOV)

- Solutions in most LLMs: **Subword tokenizer**
- Examples: Byte-Pair Encoding (BPE) tokenization, WordPiece, SentencePiece, Unigram etc.

Byte-Pair Encoding (BPE) tokenization

- BPE was initially developed for text compression.
  - First used for pretraining GPT v1, and later GPT-2, RoBERTa, BART etc.

- Basic procedure:
- Initialize a set of characters (letters and boundary symbols) from corpus.
- Iteratively **merge** the token **pairs** with the highest frequency, replacing them with new tokens.
- Repeat until a pre-defined vocab size is reached.

BPE example

- All unique English word in a corpus:
  `["loop", "pool", "loot", "tool", "loots"]`
- Initial vocab: `["l", "o", "p", "t", "s"]`, with vocab size |V|=5
- Assume the words frequencies are:

```
("loop", 15),
("pool", 10),
("loot", 10),
("tool", 5),
("loots", 8)
```

Token pairs with the highest frequency:
("o", "o") occurs 15+10+10+5+8 = 48 times

It should be merged:

("o", "o") ⇒ "oo"

Updated vocab:

`["l", "o", "p", "t", "s", "oo"]`

BPE example

- Updated corpus:

```
("l", "oo", "p", 15),
("p", "oo", "l", 10),
("l", "oo", "t", 10),
("t", "oo", "l", 5),
("l", "oo", "t", "s", 8)
```

Highest frequency pair:
("l", "oo") occurs 15+10+8 = 33 times

It should be merged:

("l", "oo") ⇒ "loo"

Updated vocab:

```
["l", "o", "p", "t", "s",
"oo", "loo"]
```

BPE example

- Updated corpus:

```
("loo", "p", 15),
("p", "oo", "l", 10),
("loo", "t", 10),
("t", "oo", "l", 5),
("loo", "t", "s", 8)
```

Highest frequency pair:
("loo", "t") occurs 10+8 = 18 times

It should be merged:

("loo", "t") ⇒ "loot"

Updated vocab:

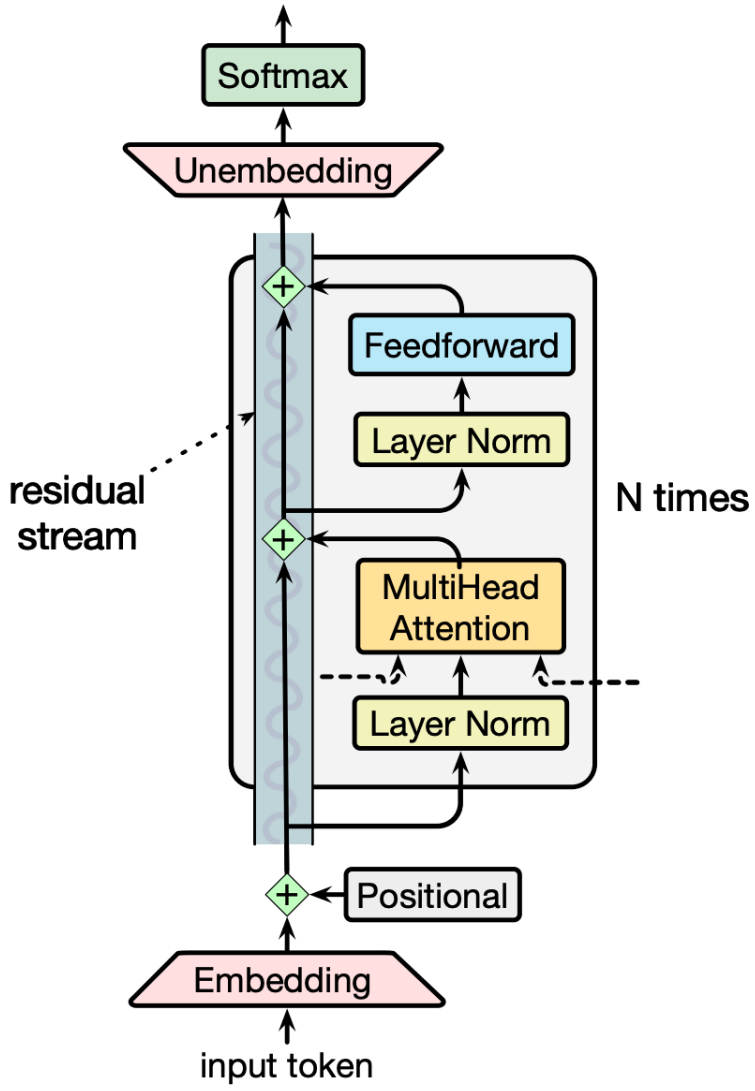
```
["l", "o", "p", "t", "s",
"oo", "loo", "loot"]
```

Current vocab size |V|=8

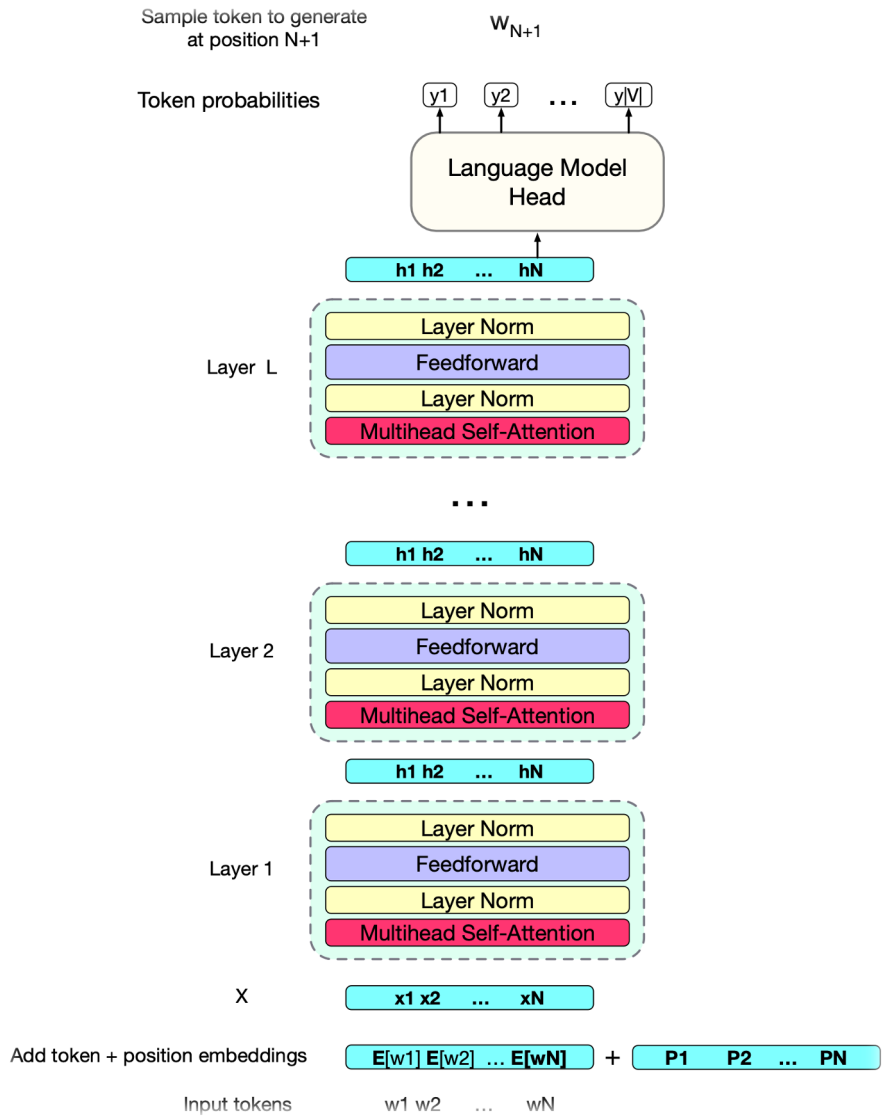
…Repeat until a certain |V| is reached

Overview

- Data
- **Model Architectures**
  - **Architecture Variants**
- Pretraining Process



Architecture terminology: Decoder-only



Transformer model has a stacked architecture of multiple layers of transformer blocks

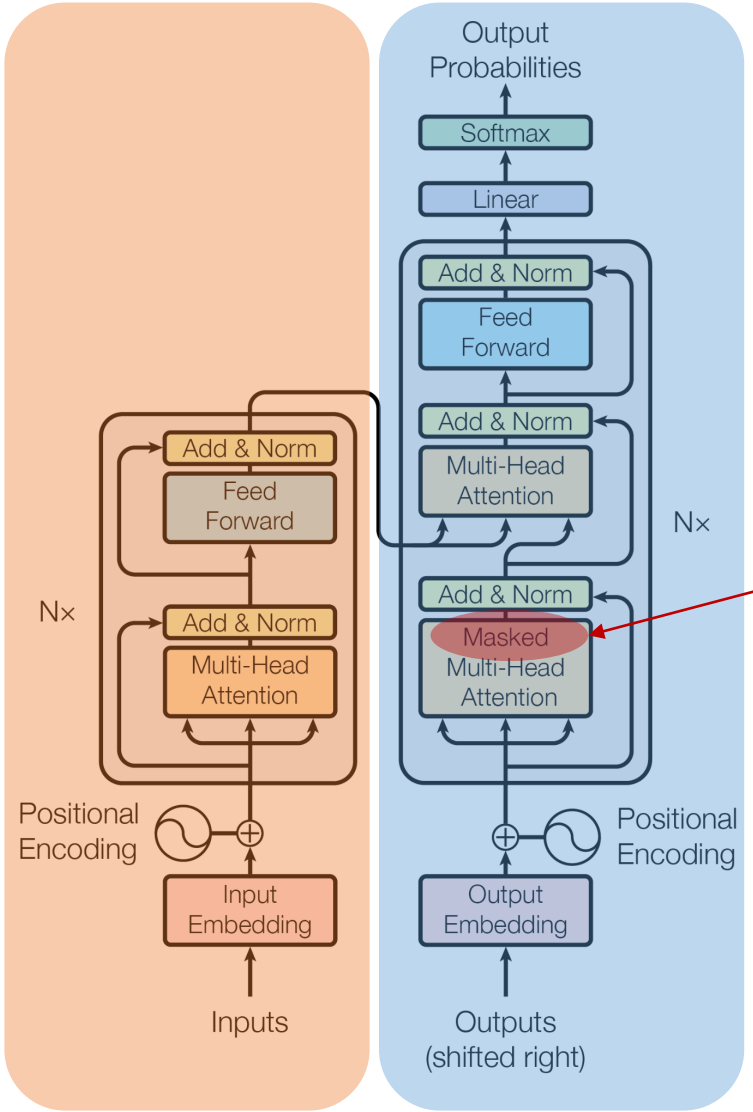
The input to all the other layers is the output $\boldsymbol{H}$ from the layer below

Terminology:

- **decoder-only model**: it is only half of the encoder-decoder model in the original work (*Attention is all you need* paper, 2017)

Architecture: Encoder-decoder

Encoder



Decoder

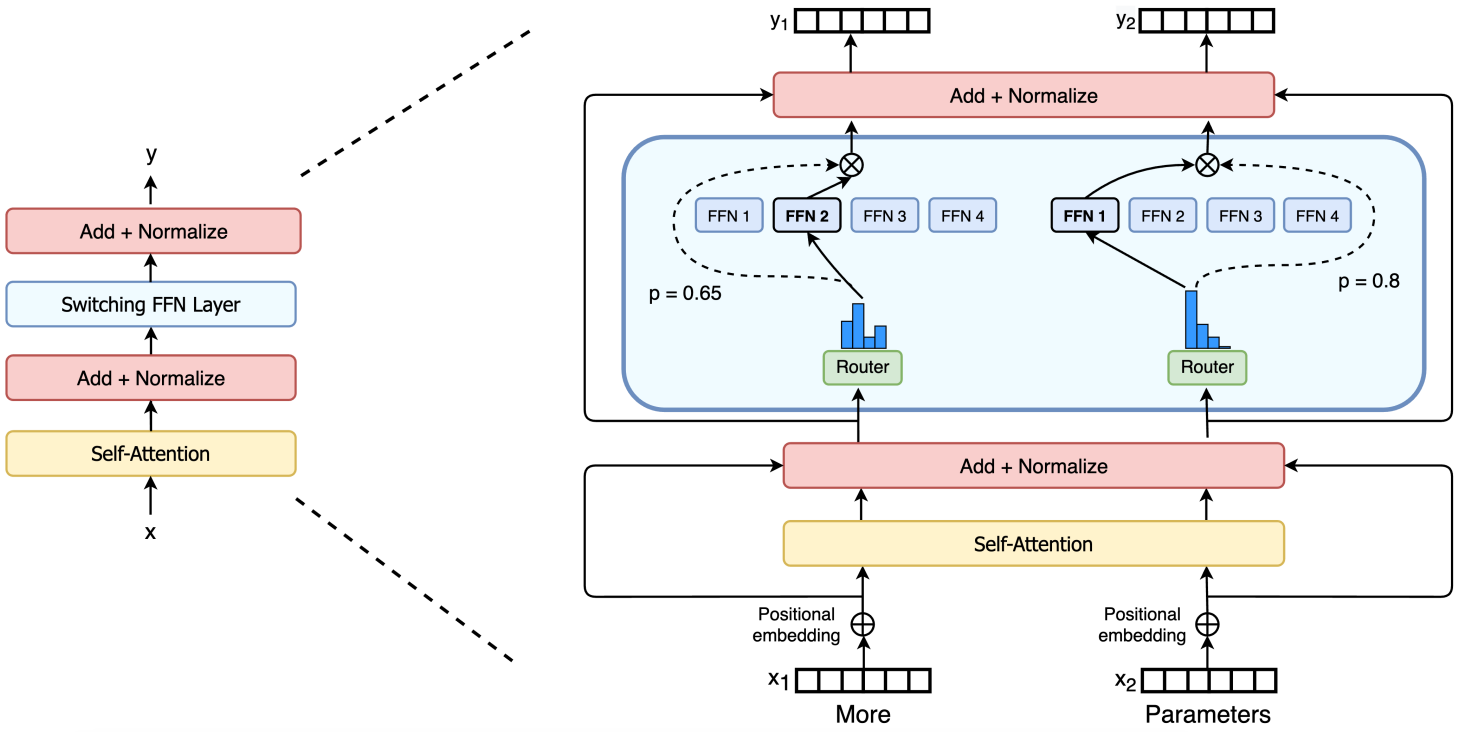
The original work is for machine translation task

- In the encoder, the self-attention is *bidirectional*;
- In the decoder, the self-attention is causal, i.e., future words are masked out

It was later that the paradigm for *causal* language model was defined using only the decoder part

Architecture: Mixture of Experts (MoE)

- How to train faster? -- Replace every FFN layer of the transformer model with an **MoE** layer



Each MoE layer has K experts:

$$E_1, E_2, \ldots, E_K$$

Each E_i is an FFN

Each token $\boldsymbol{x}_t$ is passed through a **router**, or **gate** G, computing the weight of each expert

$$G(\boldsymbol{x}_t) = \text{softmax}\Big(\text{topk}(\boldsymbol{x}_t \cdot \boldsymbol{W}^G)\Big)$$

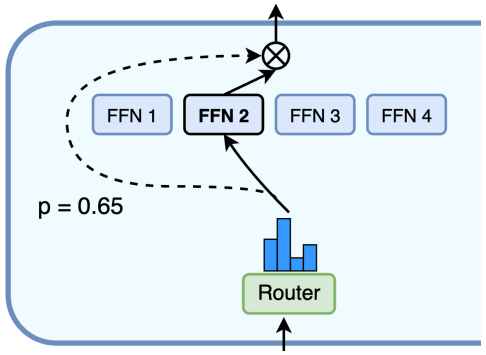
Architecture: Mixture of Experts (MoE) cont.

- Only top *k* experts will be selected (other non-selected experts will be placed with weight 0)

$$G(\boldsymbol{x}_t) = \text{softmax}\Big(\text{topk}(\boldsymbol{x}_t \cdot \boldsymbol{W}^G)\Big)$$

- $G(\boldsymbol{x}_t)$ is in the form: $[G(\boldsymbol{x}_t)_1, \ldots, G(\boldsymbol{x}_t)_k]$

- The output is the weighted sum of all selected experts



$$\text{MoELayer}(\boldsymbol{x}_t) = \sum_{i=1}^{K} G(\boldsymbol{x}_t)_i \cdot E_i(\boldsymbol{x}_t)$$

Comparison of Model Architecture

	LLaMA-3.1 (405B)	DeepSeek (67B)	DS-V2 (236B)	DS-V3 (671B)
MoE	No	No	162 experts	257 expert
# of layers	126	95	60	61
Hidden dimension d	16384	8192	5120	7168
# of attention heads	128	64	128	128

Variance on Normalization

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \qquad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2}$$

- Original transformer: Layer normalization

$$\text{LayerNorm}(\boldsymbol{x}) = \gamma\hat{\boldsymbol{x}} + \beta \qquad \hat{\boldsymbol{x}} = (\boldsymbol{x} - \mu)/\sigma$$

- More advanced methods:
- Root Mean Square Layer Normalization: **RMSNorm**

$$\text{RMSNorm}(\boldsymbol{x}) = \gamma\frac{\boldsymbol{x}}{\text{RMS}(\boldsymbol{x})} \qquad \text{RMS}(\boldsymbol{x}) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

- RMSNorm has better speed and performance
- Used in Gopher, Chinchilla, LLaMA 3.1, DeepSeek

Variance on Normalization cont.

- DeepNorm
- Scale the residual connection with some factor α

$$\mathrm{DeepNorm}(\boldsymbol{x}) = \mathrm{LayerNorm}\big(\alpha \cdot \boldsymbol{x} + \mathrm{Sublayer}(\boldsymbol{x})\big)$$

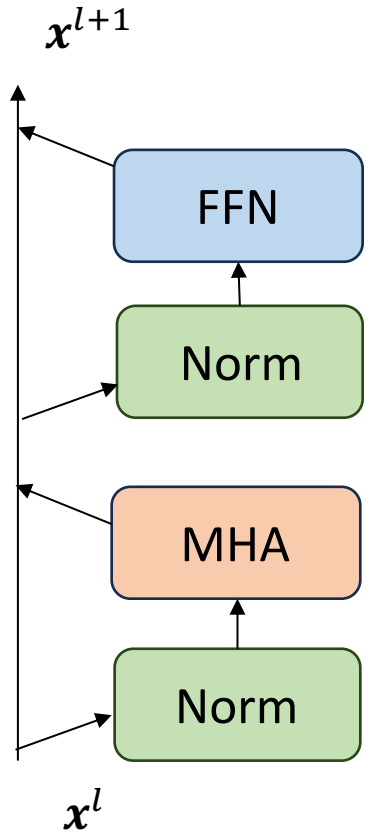
- where Sublayer represents FFN or Self-Attention (MHA)
- More robust training effect, deeper networks (>1000)
- Used in GLM-130B

Position of Normalization

- **Pre-Norm**: normalization placed before residual connection, to every sublayer

$$\text{PreNorm}(\boldsymbol{x}) = \boldsymbol{x} + \text{Sublayer}(\text{Norm}(\boldsymbol{x}))$$

- Pros: Training is more robust
- Cons: Less performant than Post-Norm

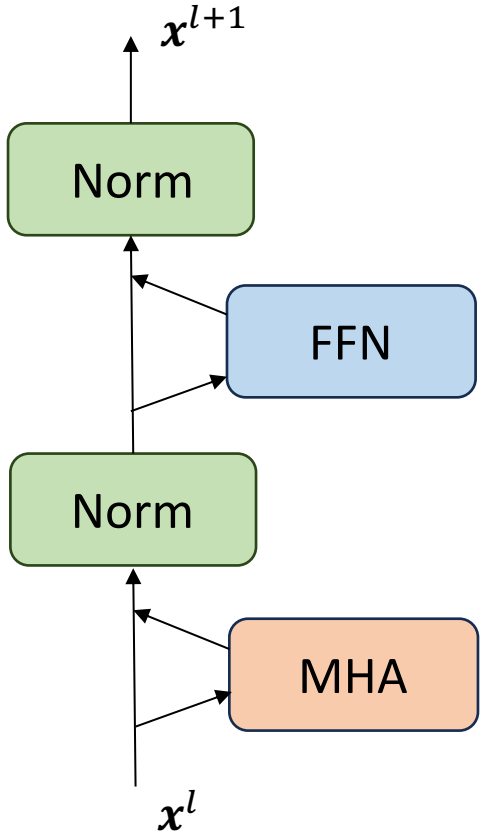


Position of Normalization

- **Post-Norm**: normalization placed after residual connection

$$\text{PostNorm}(\boldsymbol{x}) = \text{Norm}\big(\boldsymbol{x} + \text{Sublayer}(\boldsymbol{x})\big)$$

- Used in original transformer
- Pros: Training is faster to converge
- Cons: Sometimes makes training less robust



Comparison of Model Architecture cont.

	LLaMA-3.1 (405B)	DeepSeek (67B)	DS-V2 (236B)	DS-V3 (671B)
MoE	No	No	162 experts	257 expert
# of layers	126	95	60	61
Hidden dimension d	16384	8192	5120	7168
# of attention heads	128	64	128	128
Normalization	Pre RMSNorm	Pre RMSNorm	Pre RMSNorm	Pre RMSNorm

Overview

- Data
- Model Architectures
- **Pretraining Process**

Train: Scaling Laws

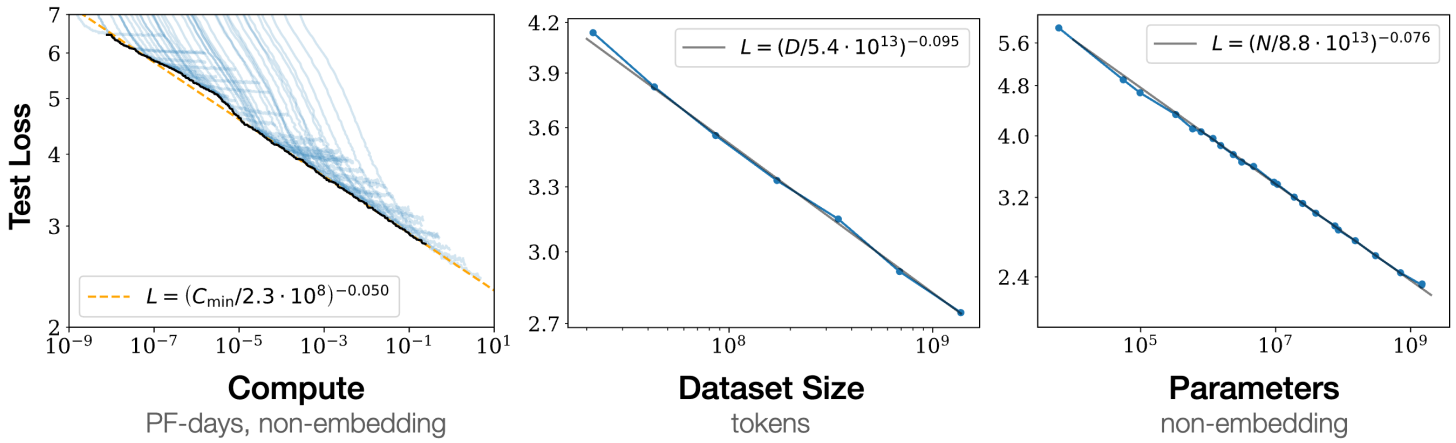
- **Scaling laws**: LLMs' performance (measured by loss) scales as a *power law* with each of the three factors: model size, dataset size, and training time

Loss L as a function of:

- Number of non-embedding parameters N
- Dataset size D
- Compute budget C

(In each case the other two factors are held constant)

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} \qquad L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} \qquad L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}$$



Ref: Kaplan et al. (2020) Scaling Laws for Neural Language Models. arxiv.org/abs/2001.08361

Training: Estimate # of Parameters

- The number of non-embedding parameters N can be estimated:
- $N \approx 2d \cdot n_{\text{layer}} \cdot (2d_{\text{attn}} + d_{\text{ff}}) \approx 12 n_{\text{layer}} d^2$
- (Assuming $d = d_{\text{attn}} = d_{\text{ff}}$)

- For GPT-3, with $n = 96$ layers and $d = 12288$, $N \approx 12{\times}96{\times}12288^2 \approx 1.74{\times}10^{11}$
- That is 174 billion parameters (千亿)